Software Design Document

– Chess Game –

1. Introduction and Purpose

The System Under Development (SUD) is a programmed chess application that allows a user to play against an opponent. The opponent can either be a real-life player or an Artificial Intelligence (AI) bot powered by an Application Programming Interface (API). The SUD is intended to allow players to view a fully visualized chess board and pieces and interact with them to compete in a game of chess following standard chess rules.

1.1 Acronyms, Terms and References

Acronym	Meaning
HDD	High-Level Design
LLD	Low-Level Design
SDD	Software Design Document
SUD	System Under Development
SWEBOK	Software Engineering Body of Knowledge
GUI	Graphical User Interface
API	Application Programming Interface
AI	Artificial Intelligence
FEN	Forsyth-Edwards Notation

Term	Definition
Forsyth-Edwards Notation	Standard Notation to describe the position of a chess game in software(see FEN reference below)

Environment Variable	Variables stored at an operating system level, outside of the source code itself (see Dagster's explanation below)

Reference	Source
FIDE Laws of Chess	https://www.fide.com/FIDE/handbook/LawsOfChess.pdf
FEN Reference	https://www.chess.com/terms/fen-chess
Pygame Documentation	https://www.pygame.org/docs/
Python 'dotenv' Package	https://pypi.org/project/python-dotenv/
Dagster's Explanation of Environment Variables in Python	https://dagster.io/blog/python-environment-variables

2 Design Considerations

2.1 Assumptions and Dependencies

- *Currently, the API described in [SRD_ESR_01] is not specified and can be changed. Currently in the design, it is the Stockfish API that is freely available on the internet.*
- *Currently, the timed requirements for [SRD_PR_01] and [SRD_PR_02] that requires the system to run at short times of 100 milliseconds for processing user inputs and 2 seconds to calculate the chess engines are ambitious and may be subject to change due to the hardware that may be running the SUD, however an assumption can be made that seeable lag will be reduced to avoid user frustration.*
- *Currently, the SUD is supposed to run using FIDE Laws of Chess and should not be skewing from that assumption*

- *Currently, the SUD does not have a save and reopen file requirement in the SRD, however, that is a planned module for the design of the SUD.*
- *Currently. The SUD does not have a difficulty selection for the API. However, due to "customer questioning" a difficulty selection module will most likely be present in the SUD.*

2.2 Design Constraints

- *Currently, the requirement [SRD_DIC_01] requires that the programming language Python must be used to program the SUD. The specific type of Python language hasn't been chosen yet but has been left open to create less constraint on the SUD.*
- *The SUD requires the installation of additional Python packages in order to support graphical rendering and environment configuration. These dependencies may include libraries such as Pygame for graphical interface functionality and python-dotenv for secure environment variable management.*
- *Currently, requirement [SRD_ESR_01] states that the SUD must communicate with an external API. This creates a dependency on the service of the API to be active to have that feature working.*
- *The SUD requires a valid API key to communicate with the external chess engine API. If valid credentials are not available, the AI opponent functionality will not operate.*
- *Currently, the SUD must be developed in a short time which may cause extra ideas to be cut out and not defined in the SRD, such as a feature to save and reopen a chess game.*
- *Currently, the SUD is constrained to being developed using school resources which can be slow and aged due to the volume of use or lack of replacement.*

2.3 Goals and Guidelines

- *The SUD is being developed to be able to choose between a two-person local play and an API versus player play. This feature is a main component of the interface for the design.*
- *The SUD is being developed to have no foreseen lag by the user by having a very small time for user inputs to be recognized and reflected from the SUD to the user.*
- *The SUD is being developed to have the computer to be able to give the user the idea that the SUD is thinking of a next move.*

- *The SUD is being developed to have low coupling between different modules of the SUD.*
- *The SUD is being developed to have high cohesion in each module of the SUD.*
- *The SUD is being developed with modularity to accommodate the two different playing styles that the user may choose.*
- *The SUD is being developed with partition to volatility to ensure the program can easily accommodate new and expected changes to be made to the SUD.*

2.4 Development Methods

Forsyth-Edwards Notation(FEN) shall be used to store the board state as a string for easy parsing and transfer of all relevant data between all modules. See above reference for a full summary of the notation style. The FEN notation of the starting game state is "rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0 1". We will also use a custom numbering index for all 64 squares on the board starting with a8 as 0 and ending with h1 as 63. The numbers from the view of white are displayed below:

0	*1*	*2*	*3*	*4*	*5*	*6*	*7*
8	*9*	*10*	*11*	*12*	*13*	*14*	*15*
16	*17*	*18*	*19*	*20*	*21*	*22*	*23*
24	*25*	*26*	*27*	*28*	*29*	*30*	*31*
32	*33*	*34*	*35*	*36*	*37*	*38*	*39*
40	*41*	*42*	*43*	*44*	*45*	*46*	*47*
48	*49*	*50*	*51*	*52*	*53*	*54*	*55*
56	*57*	*58*	*59*	*60*	*61*	*62*	*63*

3 High-Level Design (HLD)

The high-level design stage focuses on identifying the SUD's primary computational elements and significant relationships among them, with a focus on each major component's existence, role and interfaces.

Further, it specifies the interaction of the SUD's major components with one another and with the environment, including users, devices and other systems.

The HLD stage addresses the following:

- *External events and messages to which the system must respond*
- *Events and messages which the system must produce*
- *Specification of the data formats and protocols for events and messages*
- *Specification of the ordering and timing relationships between input events and messages, and output events and messages*
- *Tracing and analysis of end-to-end transactions and event threads*
- *Data persistence (how data is stored and managed)*

The resulting HLD should be sufficiently detailed to allow low-level design activities to proceed.

High-level design is undertaken within the envelope established by the system's software architecture (if any). Each of the above may be guided or constrained by architecture directives. For example, event signaling and messaging will use the protocols and modes of interaction established by the architecture. Data formats and protocols will use data and communication standards specified by the architecture. Absent an explicit architecture design stage, some of these directives will be established by the software requirements or decided during high-level design.

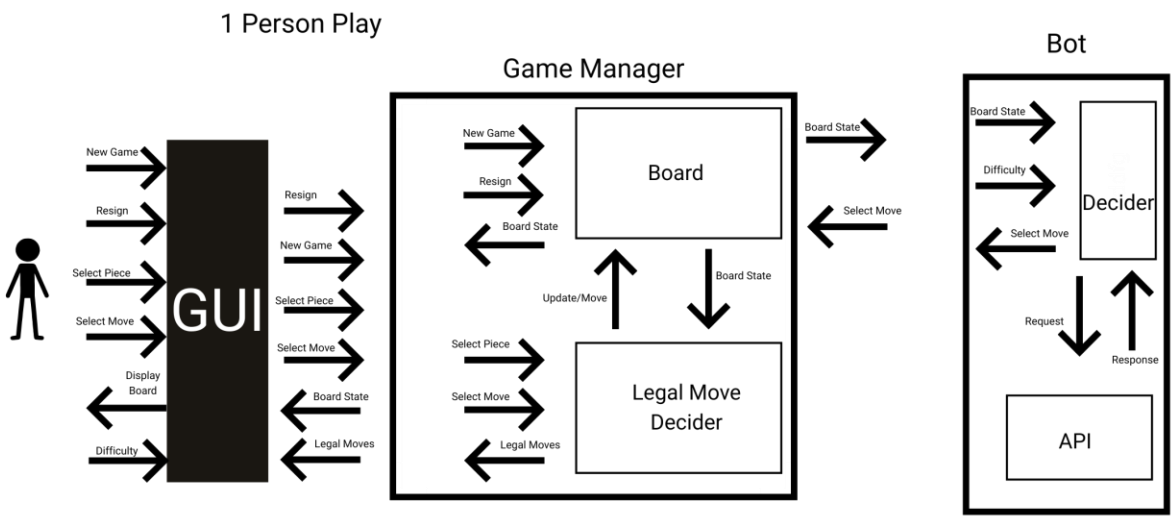

Figure 1 – 1 player Chess User Case View

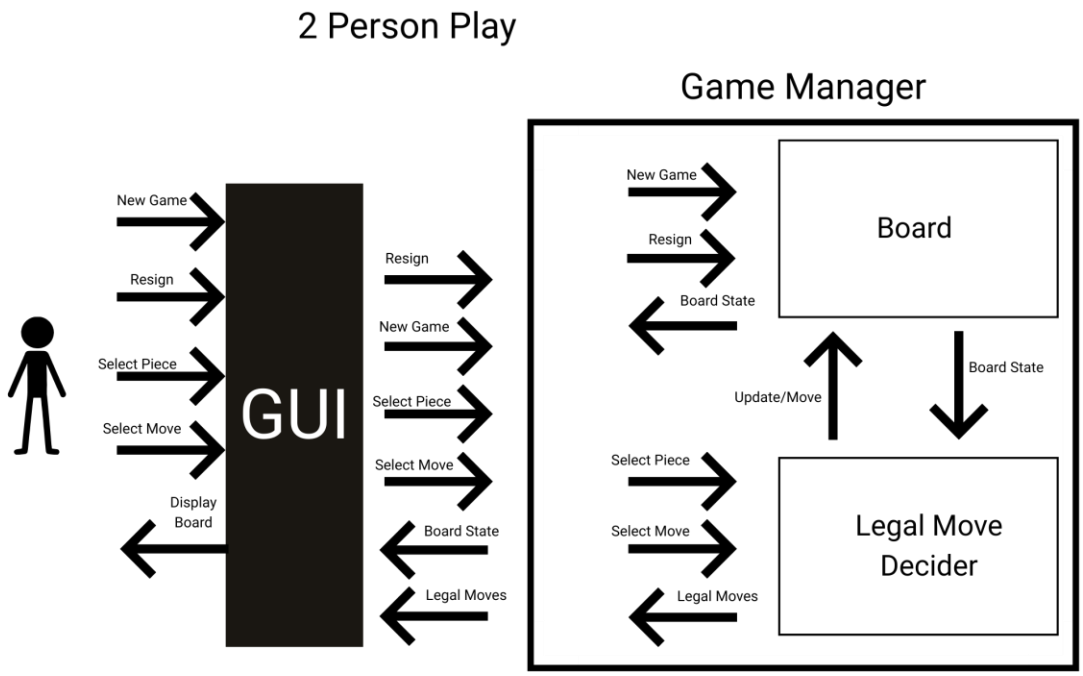

Figure 2 – 2 player Chess User Case View

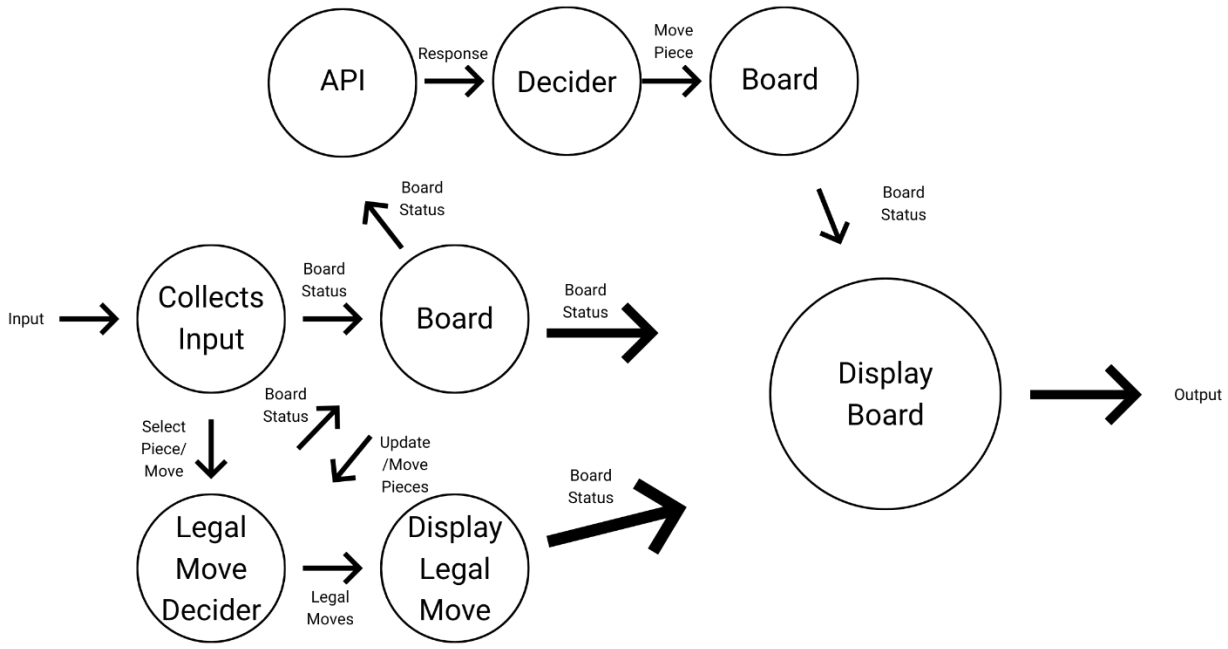

Figure 3 – Functional Diagram

3.1 GUI Design

The purpose of the GUI is to display the board to the user and handle all user inputs, which are starting a game, conceding a game, selecting a piece to move, and selecting a move for the piece. The design features are as follows:

- *[SDD_HLD_GUI_01]: The GUI shall display the current board state to the user.*
- *[SDD_HLD_GUI_02]: The GUI shall take user input to start a new game and pass it along to the Game Manager.*
- *[SDD_HLD_GUI_03]: The GUI shall take user input to resign from a game and pass it along to the Game Manager.*
- *[SDD_HLD_GUI_04]: The GUI shall take user piece selection and pass it along to the Game Manager.*

- *[SDD_HLD_GUI_05]: The GUI shall take user move selection and pass it along to the Game Manager.*

3.2 Game Manager Design

The purpose of the Game Manager is to store the board state and process moves made by the user and bot. The Game Manager has two subsections; the Board and Legal Move Decider. The Board Subsection is responsible for storing the board state and updating it when given new moves. The Legal Move Decider is responsible for returning all legal moves for a selected piece, verifying that an attempted move is legal, and then passing it on to the board. The requirements for the Game Manager are as follows:

- *[SDD_HLD_GM_01]: When a new game command is received, the Game Manager shall set the board to its initial position as outlined in FIDE 2.3.*
- *[SDD_HLD_GM_02]: When a resignation command is received, the Game Manager shall update the game state accordingly.*
- *[SDD_HLD_GM_03]: The Game Manager shall send the current board state to the GUI.*
- *[SDD_HLD_GM_04]: When a piece selection is received, the Game manager shall use the Legal Move Decider to determine all legal moves for that piece and send that list to the GUI.*
- *[SDD_HLD_GM_05]: When a move selection is received, the Game Manager shall use the Legal Move Decider to verify its legality according to FIDE Article 3.*
    - *[SDD_HLD_GM_05a]: If a move is determined to be legal, the board shall be updated accordingly.*
    - *[SDD_HLD_GM_05b]: If a move is determined to be illegal, the board shall not be updated.*
- *[SDD_HLD_GM_06]: The Game Manager shall send the board state to the Bot.*

3.3 Bot Design

The purpose of the Bot is to generate moves from the computer-driven opponent when the user chooses to play against the AI. The Bot communicates with the selected AI chess engine through an external API. The bot receives the current board state from the Game Manager in FEN format and sends this information to the API. The API evaluates the position and returns a move for the Bot to play.

The move returned by the API is determined according to the difficulty level selected by the user, allowing the system to provide different levels of challenge during gameplay.

- *[SDD_HLD_BD_01]: If the user chooses to play against a Bot, the system shall prompt the user to select a pre-determined difficulty level.*
- *[SDD_HLD_BD_02]: The selected difficulty level shall influence the move generated by the AI engine, resulting in different levels of challenge for the user.*
- *[SDD_HLD_BD_03]: The bot shall send the current board state in FEN format to the external AI through the use of an API.*
- *[SDD_HLD_BD_04]: The bot shall return the move generated by the AI engine, and return it to the Game Manager.*

4 Low-Level Design (LLD)

The low-level design stage focuses on defining each module's internal structure, focusing on detailing and justifying choices of algorithms, data access and data representation.

The detailed design stage proceeds within the constraints established by the HLD. It specifies major system components' internal characteristics, internal modules and their interconnections to other modules, services and processes they provide, computing properties, algorithms, and data access rules and data structures.

The LLD stage addresses the following:

- *Refinement of major system components into modules or program units, including opportunities for using off-the-shelf components and application frameworks*
- *Allocation of design responsibilities to modules and program units*
- *Interactions among modules*
- *Scope and visibility among components, modules and program units*
- *Component modes, component states and transitions among them*
- *Data and control interdependencies*
- *Data organization, packaging and implementation*
- *User interfaces*
- *Requisite algorithms and data structures*

The resulting LLD should be sufficiently detailed to allow implementation activities to proceed.

4.1 GUI Design

The GUI module is responsible for displaying the chess board to the user and capturing user interactions with the system. The GUI will be implemented using the Pygame library, which provides functionality for rendering graphical elements and capturing user input events such as mouse clicks and mouse coordinates.

The GUI shall operate on finite state machine-based logic with 3 states. The first state is a 'Menu' state where the players cannot interact with the board but only the elements to start a new game.

The next state is a 'Piece Selection' state where the player can interact with the board as well as menu options to resign the current game. When a player clicks on the board in this state, the mouse coordinates are converted to a board square index. The GUI sends that index to the GM and receives a list of all legal moves possible for that piece and then transitions to the next state. If the list is empty, it remains in the current state and waits for the next mouse input.

The third state is a 'Move Selection' state where the GUI additionally renders dots over all squares with a legal move for the currently selected piece. If the player clicks on one of these squares, the GUI sends that move to the GM and returns to the piece selection state. If the player clicks on a square that is not a legal move, no move is sent to the GM and the GUI returns to the piece selection state.

4.2 Game Manager Design

Given the HLD for the Game Manager laid out in Section 3.2, we derive the following LLD for it in pseudocode:

```
class Game_Manager is
  private:
    [[SDD_LLD_GM_01]]
    board //probably a string for FEN position
  public:
    [[SDD_LLD_GM_02]]
    new_game() //reset the board
        board = “rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq -
              0 1”
    [[SDD_LLD_GM_03]]
    resign()
        //trigger game end via resignation
    [[SDD_LLD_GM_04]]
    get_legal_moves(int piece) //return list of moves for a piece
        //find piece on respective square
        //calculate all legal squares to move to
        //return list of all legal moves
    [[SDD_LLD_GM_05]]
    move(int piece, int move) //make a move, checking that it’s valid
              first
        //verify that move is legal
        //make move if legal, error if not
    [[SDD_LLD_GM_06]]
    get_board() //return board state, likely FEN string
        return board
```

4.3 Bot Design

Given the HLD for Bot Design laid out in Section 3.3, we derive the following LLD for it in Pseudocode:

```
Class Bot:

      Private:

      [[SDD_LLD_BD_01]]

            difficulty_level // Determined by the user

            nthBM                  // nthBM stands for Nth-best move.
      will be set depending on determined difficulty. Can equal an
      int 1 thru 5

      Public:

      [[SDD_LLD_BD_02]]

      set_difficulty(difficulty):

      // set bot parameters based on user-selected difficulty

      difficulty_level = difficulty

      nthBM = // set to a int

      // Initialize internal parameters such as depth, runtime, elo


      [[SDD_LLD_BD_03]]

      generate_move(board_fen)

      // send board state to the API in FEN using api_key

      // received list of best moves

      // select a move based randomly from 1 through N best moves

// return selected move to game manager
```

4.4 API Key and Environment Configuration

To protect sensitive credentials required for communicating with the external chess engine API, the SUD will store API authentication information in an environment configuration file rather than directly in the source code. The environment configuration file (.env) will contain the API key required to authenticate requests to the external AI. Additionally, the SUD will leverage Python's python-dotenv package to load the .env file at runtime. Rather than exposing the access credentials in the source code repository, this approach allows the application to be configured across development environments without modifying program logic. The .env file will not be included in the source control repository and will be instead loaded at runtime using a configuration management library.

- *[[SDD_LLD_API_01]] The system shall store external API authentication credentials in a .env configuration file.*
- *[[SDD_LLD_API_02]] The SUD shall load environment variables from .env file during program initialization.*
- *[[SDD_LLD_API_03]] The system shall retrieve the API key from the environment configuration prior to making requests to the external chess engine API.*
- *[[SDD_LLD_API_04]] The SUD shall prevent the .env configuration file from being committed to the source code repository.*
- *[[SDD_LLD_API_05]] If the API key is missing or invalid, the system shall generate an error message and disable Bot opponent functionality.*

The following example demonstrates the configuration of the .env file:

```
# example .env file configuration structure


API_KEY = //API key or credentials pasted here


API_URL = //paste URL of intended chess engine API here
```

4.4.1 Integration with Bot Module

During system initialization, the Bot module will load the .env file and retrieve the API key before communicating with the external API service. The following pseudocode demonstrates how the Bot module integrates environment configuration:

```
Environment Configuration Module


[[SDD_LLD_API_01]]
Configuration initialization:
      import dotenv library
      import operating system environment library


      //load environment variables from ".env" file
      load_dotenv()


[[SDD_LLD_API_02]]
//Retrieve API configuration:
      api_key = getenv("API_KEY")
      api_url = getenv("API_URL")


[[SDD_LLD_API_03]]
Validate configuration:
      if api_key is null or empty:
            //display error message
            //disable Bot functionality
      else:
            //continue to set_difficulty()
            continue


[[SDD_LLD_API_04]]
//return configuration values to modules that require API access (Bot
Class)
```

4.4.2 Development Environment Setup Example

To support graphical rendering, API communication, and environment configuration, the development environment must include several Python packages. These dependencies must be installed prior to executing the SUD.

The following example demonstrates the installation of required packages using Python's package manager:

```
//to be executed in terminal, outside of source code file(s)


pip install pygame //required for graphical rendering of chess board
and handling user input for GUI design


pip install python-dotenv //used to load environment variables from
.env configuration file


pip install requests //required for communication with the external
chess engine API
```

5 Trace Tags

List the trace tags defined in the SDD and their mapping(s) to requirements in the SRD.

- *[SDD_HLD_GUI_01]::[SRD_FL_01], [SRD_IR_02], TBD*
- *[SDD_HLD_GUI_02]::[SRD_FL_02], [SRD_UR_01], [SRD_IR_03], TBD*
- *[SDD_HLD_GUI_03]::[SRD_IR_05], TBD*
- *[SDD_HLD_GUI_04]::[SRD_FL_02], [SRD_IR_02], TBD*
- *[SDD_HLD_GUI_05]::[SRD_FL_02], [SRD_IR_02], TBD*
- *[SDD_HLD_GM_01]::[SRD_FR_01], TBD*
- *[SDD_HLD_GM_02]::[SRD_UR_05], [SRD_UR_06], TBD*

- *[SDD_HLD_GM_03]::[SRD_FL_01], TBD*
- *[SDD_HLD_GM_04]::[SRD_FR_02], [SRD_FR_03], [SRD_FR_04], [SRD_FR_05], [SRD_FR_06], [SRD_FR_07], TBD*
- *[SDD_HLD_GM_05]::[SRD_FR_02], [SRD_FR_03], [SRD_FR_04], [SRD_FR_05], [SRD_FR_06], [SRD_FR_07], TBD*
- *[SDD_HLD_GM_05a]::[SRD_FL_02], TBD*
- *[SDD_HLD_GM_05b]::[SRD_UR_04], TBD*
- *[SDD_HLD_GM_06]::[SRD_IR_01], [SRD_IR_06], [SRD_ESR_01], TBD*
- *[SDD_LLD_GM_01]::[SDD_HLD_GM_01], [SDD_HLD_GM_03], [SDD_HLD_GM_06], [SRD_FL_01], [SRD_FR_01]], TBD*
- *[SDD_LLD_GM_02]::[SDD_HLD_GM_01], [SRD_UR_01], [SRD_FR_01], [SRD_FL_02]], TBD*
- *[SDD_LLD_GM_03]::[SDD_HLD_GM_02], [SRD_IR_05], [SRD_UR_05], [SRD_UR_06]], TBD*
- *[SDD_LLD_GM_04]::[SDD_HLD_GM_04], [SDD_HLD_GM_05], [SRD_FR_02], [SRD_FR_03], [SRD_FR_04], [SRD_FR_05], [SRD_FR_06], [SRD_FR_07]], TBD*
- *[SDD_LLD_GM_05]::[SDD_HLD_GM_05], [SDD_HLD_GM_05a], [SDD_HLD_GM_05b], [SRD_FL_02], [SRD_UR_03], [SRD_UR_04], [SRD_FR_02], [SRD_FR_03], [SRD_FR_04], [SRD_FR_05], [SRD_FR_06], [SRD_FR_07]], TBD*
- *[SDD_LLD_GM_06]::[SDD_HLD_GM_03], [SDD_HLD_GM_06], [SRD_FL_01], [SRD_IR_06]], TBD*
- *TBD*

Appendix

A – FEN Notation Explanation

Forsyth–Edwards Notation (FEN) is a standardized method used to represent a specific position in a chess game using a single line of text. Instead of storing the board as a graphical diagram or a long list of moves, FEN provides a compact way to encode all of the information necessary to reconstruct the exact state of a game at a particular moment. This format is widely used in chess software, databases, and engines because it allows positions to be easily saved, shared, and analyzed by computer programs. By reading a FEN string, a chess engine or application can immediately recreate the board and determine what moves are legal from that position.

A FEN string consists of six fields separated by spaces, each containing different information about the game. The first field describes the piece placement on the board. The board is written rank by rank starting from rank 8 (Black's back rank) down to rank 1 (White's back rank). Within each rank, pieces are represented by letters: uppercase letters represent White's pieces and lowercase letters represent Black's pieces. The letters correspond to the piece names (`K`/`k` for king, `Q`/`q` for queen, `R`/`r` for rook, `B`/`b` for bishop, `N`/`n` for knight, and `P`/`p` for pawn). Empty squares are represented by numbers indicating how many empty squares occur consecutively in that rank. Slashes (`/`) are used to separate the ranks.

The second field indicates which player has the move, using `w` for White and `b` for Black. The third field records castling availability, showing which sides still retain the right to castle. This is represented by the letters `K` and `Q` for White's kingside and queenside castling rights, and `k` and `q` for Black's. If neither side can castle, a dash (`-`) is used instead. The fourth field represents the en passant target square, which is the square where an en passant capture would be possible on the next move if one exists. If there is no en passant opportunity, this field is also written as `-`.

The final two fields provide information related to move counting. The halfmove clock records the number of halfmoves (individual player moves) since the last pawn move or capture, which is used to enforce the fifty-move rule. The fullmove number counts the number of complete moves in the game and increases after Black's move. Together, these fields ensure that the game state can be resumed accurately from the position.

For example, the starting position of a chess game is represented by the FEN string:

```
rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0 1
```

This notation indicates the initial arrangement of pieces, that it is White's turn to move, that both sides still retain all castling rights, that there is no en passant target square, and that the game is at the beginning with zero halfmoves played and the first full move about to begin. Because FEN contains all the information necessary to reconstruct the board and game state, it serves as a fundamental tool in chess programming and analysis. Chess engines, databases, and graphical interfaces frequently rely on FEN to load positions quickly, test algorithms, and store positions without needing to record the entire history of moves that produced them.